AI-INTEGRATED MULTI-MODAL ACADEMIC EVALUATION, PERFORMANCE INTELLIGENCE, AND AUTOMATED VIVA ASSESSMENT SYSTEM

INDIVIDUAL THESIS

Liyanage Supun Buddhika Hemarathne

(IT22134776)

Individual component: AI-Driven Exam Question Analytics and Prediction Module

Group ID: R26-SE-025

Report submitted in partial fulfilment of the requirements for the

B.Sc. (Hons) in Information Technology

Specializing in Software Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Liyanage Supun Buddhika Hemarathne

Student ID: IT22134776

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Ms. Sanjeevi Chandrasiri		
Co-supervisor: Ms. Shalini Rupasinghe		

Abstract

This thesis examines an AI-assisted exam analytics and practice recommendation component within an integrated academic evaluation system. The component addresses the gap between identifying learning weaknesses from assessment evidence and selecting relevant follow-up practice. An artifact-centred case study investigates AdaptiveExamAnalyticsEngine, which combines a ModernBERT classifier for revised Bloom levels, deterministic assessment analytics, keyword-based topic assignment, and a transparent five-signal recommendation rule. Evaluation uses an eleven-question temporal repository fixture covering six Bloom classes and a ranking corpus of 100 candidates across ten synthetic weakness profiles, assessed by two project-affiliated raters. The classifier achieves accuracy and macro-F1 of 1.000 on the small fixture, which has no exact normalized overlap with the 8,767-question training CSV. These findings establish limited integration evidence rather than broad generalization. Default precision at five is 0.360 and 0.320 for the two raters, while normalized discounted cumulative gain at five is 0.420 and 0.414. A weakness-only baseline matches precision and has slightly higher ranking-gain point estimates. Paired bootstrap intervals cross zero, so the additional recommendation signals do not establish incremental value. Question-bank imbalance, topic-label errors, and inconsistent rater interpretation of difficulty constrain the findings. The principal contribution is a documented evidence-to-action architecture and a baseline-driven evaluation that identifies conditions required for more credible educational validation. The implemented component supports practice selection and lecturer decision-making; it does not forecast future examination questions or establish improved student learning outcomes.

Keywords: learning analytics, Bloom taxonomy, exam analytics, practice recommendation, repository case study

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my research supervisor for their continuous guidance, encouragement, and valuable feedback throughout the development of this research proposal. Their support and expertise have greatly contributed to shaping the direction and quality of this study. I would also like to thank the faculty and academic staff of the Department of Information Technology at SLIIT for providing the necessary academic environment and resources required for conducting this research. Furthermore, I would like to extend my appreciation to my team members for their collaboration, support, and dedication during the preparation of this project proposal. Finally, I would like to thank my family and friends for their constant encouragement and support throughout my academic journey.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i
Abstract ii
ACKNOWLEDGEMENTS iii
TABLE OF CONTENTS iv
LIST OF TABLES vi
LIST OF ABBREVIATIONS vii
1 INTRODUCTION 1
1.1 Overall Project Context 1
1.2 Individual Component Context 1
1.3 Problem Statement 2
1.4 Research Gap 3
1.5 Aim And Objectives 3
1.6 Research Questions Or Hypotheses 4
1.7 Scope Limitations and Assumptions 4
1.8 Individual Contribution 5
1.9 Thesis Structure 5
2 LITERATURE REVIEW AND RELATED WORK 6
2.1 Review Method and Scope 6
2.2 Domain and Theoretical Foundations 6
2.3 Related Research 7
2.4 Existing Systems Products and Techniques 8
2.5 Gap Synthesis and Conceptual Framework 8
3 RESEARCH METHODOLOGY 9
3.1 Research Design 9
3.2 Data Participants or Study Materials 9
3.3 Experimental or Study Procedure 10
3.4 Measures and Evaluation Metrics 10
3.5 Data Analysis 11
3.6 Ethics Privacy Security and Responsible AI 12
3.7 Reproducibility and Research Artefacts 12
4 PRODUCT AND SOLUTION DESIGN 14
4.1 Requirements 14
4.2 Component Architecture Within the Complete Solution 14
4.3 Design Alternatives and Decisions 15
4.4 Detailed Design 16
4.5 Standards and Professional Practice 18
4.6 Commercialisation and Sustainability 18
5 IMPLEMENTATION INTEGRATION AND TESTING 20
5.1 Development Environment and Technology Stack 20
5.2 Component Implementation 20
5.3 Integration with the Other Components 21
5.4 Verification and Software Testing 22
5.5 Deployment and Operation 23
6 RESULTS AND VALIDATION 24
6.1 Research Results 24
6.2 Product And Technical Results 24
6.3 User Or Stakeholder Validation 25
6.4 Integration Results 26
6.5 Summary of Findings 27

6.6 Supplementary Repository Verification 27
7 DISCUSSIONS 29
7.1 Interpretation of Findings 29
7.2 Comparison with Literature Baselines and Existing Solutions 29
7.3 Research Contribution 30
7.4 Product Contribution 30
7.5 Limitations and Threats to Validity 31
7.6 Implications 32
8 CONCLUSION AND FUTURE DIRECTIONS 33
8.1 Conclusion 33
8.2 Achievement of Objectives 33
8.3 Individual Research And Product Contributions 34
8.4 Future Directions 34
REFERENCES 36
GLOSSARY 38
APPENDICES 39
Appendix A Evaluation Evidence Record 39
Appendix B Academic Integrity and AI Use Disclosure ..... **Error! Bookmark not defined.**

LIST OF TABLES

Table 6.1 Temporal fixture confusion matrix with gold labels in rows 24
Table 6.2 Ranking comparison over ten synthetic weakness profiles 25
Table 6.3 Question bank distribution by source and Bloom level 27

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial intelligence
API	Application programming interface
BT	Bloom taxonomy level
CSV	Comma-separated values
DBMS	Database management systems
F1	Harmonic mean of precision and recall
JDBC	Java Database Connectivity
LLM	Large language model
NDCG	Normalized discounted cumulative gain
P@5	Precision at five
SQL	Structured Query Language

1 INTRODUCTION

1.1 Overall Project Context

The overall project, AI-Integrated Multi-Modal Academic Evaluation, Performance Intelligence, and Automated Viva Assessment System, concerns the use of assessment evidence to support academic evaluation. Within this context, the individual component examined in this thesis connects graded examination evidence to an interpretable profile of learning needs and a set of recommended practice questions. The group project title establishes the wider context, while the present evaluation is limited to the exam analytics and practice recommendation component.

Assessment records contain information that a final mark alone cannot express. Two students may obtain similar overall marks while having different weaknesses across topics or cognitive processes. At cohort level, a low score on a question may indicate a topic that requires further teaching, an unsuitable question, or a mismatch between the assessment and prior instruction. These interpretations require context and human judgement. Learning analytics provides a basis for organizing such evidence into information that can support educational decisions [1].

The component's intended role is to help lecturers inspect this evidence and identify appropriate follow-up activities. Its outputs remain decision support. Neither a computed weakness score nor a generated question replaces the lecturer's responsibility to review subject correctness, curriculum alignment, and suitability for a particular learner.

1.2 Individual Component Context

The individual component is formally identified in the project details as the AI-Driven Exam Question Analytics and Prediction Module. The evaluated implementation is named AdaptiveExamAnalyticsEngine. The research paper establishes a narrower implemented scope: the engine analyses completed assessment evidence and recommends weakness-aligned practice; it has no time-series, recurrence, or probabilistic model for predicting future examination content

[2]. Throughout this thesis, prediction-oriented wording in the registered component title is therefore distinguished from the functions actually evaluated.

The engine accepts course metadata, rubrics, graded submissions, and a heterogeneous question bank. It assigns a revised Bloom level to question text, maps questions to canonical database-management topics, aggregates awarded and maximum marks, and derives weakness measures. A fixed scoring rule then ranks candidate practice items. Optional local large language model generation can produce follow-up questions or teaching actions after ranking [2].

The case study concerns a database management systems (DBMS) course. Its question bank contains 179 records, and its recommendation evaluation uses ten constructed weakness profiles. This course-specific setting permits a concrete inspection of the complete pipeline while limiting claims about transfer to other disciplines.

The local source inspected for this thesis is identified separately as a component implementation snapshot [3]. It supplies additional detail about caching, course-specific taxonomies, APIs, and shared-client integration. These implementation observations extend the system description without assuming that every current feature was part of the earlier experimental setup.

1.3 Problem Statement

The problem addressed is the gap between observing assessment weaknesses and selecting useful practice from available learning material. A dashboard can show low achievement in a topic without establishing which question should be attempted next. Question selection must account for the learner's observed weakness, the available curriculum material, cognitive demand, and the suitability of the item itself.

Automating this process introduces a second problem: a plausible recommendation formula may appear useful without outperforming a simpler rule. Multiple signals can make a ranking look more sophisticated while introducing parameters that lack empirical justification. A reliable evaluation must therefore examine both the functioning of the integrated system and the value of the additional ranking signals.

The research problem is to determine what an integrated, transparent exam analytics prototype can demonstrate with repository-based evidence, and to identify the technical and evaluation limitations that prevent stronger claims of educational effectiveness.

1.4 Research Gap

Adaptive educational systems and educational recommenders establish the value of modelling learner needs and contextual information [4]-[7]. Knowledge tracing provides methods for estimating changing knowledge states from sequences of learner interactions [8], [9]. These approaches do not by themselves validate the specific combination of deterministic assessment aggregation, automatic Bloom classification, and hand-set recommendation weights used in this component.

The case-study gap is empirical and architectural. The implementation requires an explicit account of how evidence becomes a recommendation, a check of the classifier across all six Bloom classes, comparison of the multi-signal ranker with simple baselines, and an explanation of disagreement between human raters. Without these checks, a successful demonstration could be mistaken for evidence of robust classification, superior ranking, or improved learning.

The study addresses this gap through system characterization and failure analysis. It does not claim that the five-signal scoring rule is a new learning algorithm or that all existing educational systems lack comparable functions. Its contribution is the evaluation of this particular artifact and the lessons that can be drawn from its constraints.

1.5 Aim And Objectives

The aim is to characterize and evaluate an AI-assisted examination analytics component that transforms graded evidence into interpretable learning-weakness profiles and practice recommendations.

The first objective is to document the component's evidence-to-action architecture and distinguish deterministic numerical processing from model-based semantic interpretation and generation. The second is to assess the deployed Bloom classifier

on a temporal repository fixture that represents all six classes and to examine exact text overlap with the available training CSV.

The third objective is to compare the default recommendation rule with random and weakness-only baselines using precision at five and normalized discounted cumulative gain at five. The fourth is to examine sensitivity to the weakness weight and uncertainty in the comparison with the simpler baseline. The fifth is to diagnose rater agreement, question-bank coverage, and implementation risks, and to define requirements for a stronger subsequent evaluation.

1.6 Research Questions Or Hypotheses

RQ1: How does the component transform graded assessment evidence into topic- and Bloom-aware analytics and practice recommendations?

RQ2: How does the deployed classifier behave on the eleven-question temporal fixture, and what limits the interpretation of its apparent accuracy?

RQ3: Does the default five-signal recommendation rule demonstrate additional value over random and weakness-only ranking within the ten-profile evaluation?

RQ4: What do weight sensitivity, rater diagnostics, and question-bank coverage reveal about the reliability and limitations of the recommendations?

These questions organize an exploratory case study. The evaluation does not test a hypothesis about improved examination scores or student learning gains, because it contains no corresponding intervention or longitudinal outcome study.

1.7 Scope Limitations and Assumptions

The scope includes a local ModernBERT question classifier, deterministic topic and assessment analytics, the fixed recommendation formula, optional semantic similarity and generation capabilities, and the repository-based evaluation described in the accompanying paper [2]. The input domain is DBMS material. The principal ranking evidence consists of perceived usefulness ratings for synthetic weakness profiles, rather than behavior observed during a real course intervention.

The study excludes exam-content forecasting, a learned ranking model, longitudinal knowledge tracing, causal measurement of educational benefit, and independent

validation across institutions. Other group components, including automated viva assessment, are contextual rather than independently evaluated contributions of this thesis.

Interpretation assumes that the reported rubric labels, rating-sheet entries, and repository records are the case-study evidence under examination. The availability of an 8,767-question training CSV supports an exact overlap check, but incomplete checkpoint-training provenance prevents that check from establishing complete freedom from data leakage.

1.8 Individual Contribution

The author designed and developed the entire AdaptiveExamAnalyticsEngine component within R26-SE-025, covering exam and student analytics, question semantics, weakness representation, practice recommendation, and associated interfaces. The author also designed the entire client-side user interface for the group project. The grading, diagram-evaluation, and viva services provide the wider integration context [3].

The author's research contribution comprises architecture characterization, the six-class classifier integration check, ranking baseline comparisons, sensitivity analysis, and interpretation of rater calibration and content coverage [2]. Together with the component implementation and client-side interface design, this work constitutes the author's individual contribution.

1.9 Thesis Structure

Chapter 2 positions the component within learning analytics, adaptive recommendation, cognitive classification, and evaluation research. Chapter 3 describes the case-study design, data, metrics, and limitations of the evaluation. Chapter 4 explains requirements, architecture, design choices, and prospective commercialization considerations. Chapter 5 describes the reported implementation and the scope of technical verification. Chapter 6 presents results, and Chapter 7 interprets their implications and threats to validity. Chapter 8 concludes by assessing the objectives and identifying priorities for future work.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

This chapter is a focused narrative review of the research areas relevant to the component. It draws on the literature identified in the accompanying research paper and organizes it by the role each area plays in the system: interpreting assessment evidence, representing learner needs, selecting resources, classifying cognitive demand, and evaluating outputs [2]. It is not presented as a systematic review with an exhaustive search or a registered selection protocol.

The comparison concerns conceptual relevance rather than a claim that all cited systems have been reproduced. In particular, a published adaptive learning system may use different data, objectives, users, or outcome measures. Those differences must be retained when discussing what the current prototype contributes.

2.2 Domain and Theoretical Foundations

The revised Bloom taxonomy distinguishes Remember, Understand, Apply, Analyze, Evaluate, and Create [10]. These categories provide a vocabulary for describing cognitive processes required by a question. In the component, they are represented as BT1 through BT6. A question's Bloom label is a semantic interpretation of its demand; it should not be treated as a direct empirical measurement of how difficult the question will be for a particular student.

Learning analytics concerns the use of learner-related evidence to support understanding and improvement of learning [1]. In this case, the evidence consists of marks, rubric criteria, question text, and course context. Aggregation produces interpretable summaries, but the educational meaning of a low score still depends on the assessment and the context in which it was completed.

A further distinction is necessary between observed achievement and latent mastery. The weakness score summarizes available assessment evidence. It is not a validated estimate of a student's underlying knowledge state. Sparse coverage, difficult assessments, inconsistent marking, or incorrect topic assignment can affect that

summary. This distinction supports the decision to present outputs for lecturer review.

2.3 Related Research

Adaptive and intelligent educational systems use learner information to alter learning content or support [4]. Context-aware recommenders extend resource selection by considering learner, resource, activity, and environmental context [5]. Ontology-based and hybrid approaches demonstrate ways to combine domain structure with other recommendation signals [6], [7]. These lines of work motivate the use of topic information and curriculum context in the examined engine.

Other educational systems illustrate different forms of learner-resource alignment. APELS addresses adaptable and personalized learning in computer science programmes [11]. ElectronixTutor combines learning resources within an intelligent tutoring environment [12]. Collaborative tagging provides another mechanism for personalized recommendation [13]. Their relevance is that recommendation requires an account of the learner and of the resources; their existence does not establish the effectiveness of the present scoring formula.

Knowledge tracing models the development of procedural knowledge through learner interactions [8], while Deep Knowledge Tracing learns temporal representations from interaction sequences [9]. The current engine instead aggregates completed assessment evidence into a current weakness snapshot. Consequently, it should be compared as a transparent assessment-driven recommendation artifact, rather than described as a temporal mastery model.

ModernBERT provides the encoder model family used by the deployed sequence classifier [14]. Sentence-BERT provides an approach to semantic sentence representations that is relevant to the optional similarity workflow [15]. Automatic question generation has a substantial educational literature, and reviews emphasize the need to assess generated content beyond successful text production [16]. Work on Bloom-guided quiz creation similarly motivates teacher involvement when using large language models [17].

2.4 Existing Systems Products and Techniques

The component combines techniques with different evidential requirements. Deterministic aggregation can be checked against known mark totals. A semantic classifier requires labelled examples and an appropriate test distribution. A ranker requires relevance judgements and comparison methods. A generator additionally requires subject and pedagogical review. Successful operation at one layer cannot validate all the others.

The evaluation uses normalized discounted cumulative gain (NDCG) to account for where relevant candidates occur in a ranked list [18]. Precision at five instead focuses on the proportion of selected items judged usable. Reporting both matters because methods can select the same number of useful questions while ordering them differently.

Cohen's kappa assesses categorical agreement beyond agreement expected from the raters' marginal distributions [19]. For ordinal judgements, the case study supplements kappa with exact agreement, within-one-point agreement, and Spearman correlation. These measurements answer different questions and help distinguish systematic scale offsets from disagreement over item ordering.

2.5 Gap Synthesis and Conceptual Framework

The conceptual framework is an assessment-to-practice cycle. Graded evidence is first associated with question semantics. Deterministic aggregation then produces topic and cognitive summaries. These summaries support a weakness profile, which is combined with curriculum and question-bank evidence to rank practice candidates. Optional generation and lecturer review convert the ranked evidence into potential instructional action.

The framework exposes two dependencies that a ranking metric alone can conceal. First, an incorrectly assigned topic can connect a question to the wrong weakness. Second, a weakly covered topic may contain no suitable candidate for the ranker to retrieve. Classification, labelling, bank coverage, ranking, and review therefore require separate attention.

3 RESEARCH METHODOLOGY

3.1 Research Design

The research follows an artifact-centred repository case study. The artifact is an existing AI-assisted exam analytics prototype; the study characterizes its implemented behavior and evaluates selected outputs. This design is suitable for identifying integration failures, empirical limitations, and discrepancies between documented and active behavior. It provides weaker support for population-level educational conclusions than an independently sampled deployment study [2].

Two main evaluation strands are used. The classifier strand examines a temporal repository fixture with six-class coverage and an exact normalized text-overlap check. The recommendation strand evaluates ranked candidates across synthetic profiles using two project-affiliated raters, simple baselines, paired uncertainty estimates, and a one-factor sensitivity analysis. The strands evaluate different units and their results are reported separately.

3.2 Data Participants or Study Materials

The question bank contains 179 records across eleven canonical DBMS topics. By source, these comprise 90 lecture items, 55 tutorial items, 18 generated items, and 16 prior-exam items. By cognitive label, 132 are Apply, 39 are Understand, and eight are Analyze. No bank records are assigned to Remember, Evaluate, or Create [2]. The six-class classifier fixture should not be confused with the narrower cognitive coverage of this bank.

The recommendation corpus contains 100 candidates arranged into ten synthetic weakness profiles, with ten candidates per profile. Two project-affiliated raters, identified as lec01 and lec02, independently complete the same rating sheet. These are project-internal usefulness judgements, not ratings from an independently recruited sample of teachers.

The deployed classifier checkpoint in the examined working copy is dated 29 August 2026. A newer rubric fixture dated 31 August contains ten DBMS questions with repository-assigned labels, including two examples for each of BT1, BT2, BT3, BT4,

and BT6. A prospectively specified BT5 item is added to obtain nominal coverage of all six classes. The available training CSV contains 8,767 questions [2].

3.3 Experimental or Study Procedure

For the classifier check, the ten existing fixture questions are supplemented with one Evaluate item concerning strict two-phase locking and snapshot isolation in a high-contention banking workload. The question asks for judgement using serializability, deadlock risk, write skew, and throughput. Its wording is specified before the additional inference, evaluated once, and not revised after observing the prediction. This procedure limits direct wording adjustment to that output, but it is not equivalent to preregistration of the overall study.

The eleven item texts are compared with the training CSV using exact normalized text matching. The reported absence of matches rules out that particular form of duplication. It does not establish the absence of paraphrases, semantic overlap, or undocumented training exposure. Predictions are compared with the repository labels to produce accuracy, six-class macro-F1, and a confusion matrix.

For recommendation evaluation, the candidate ordering produced by the default rule is compared with random ordering and a weakness-only ordering within each profile. Binary relevance is the raters would-use decision. The random baseline averages 10,000 fixed-seed permutations per profile. The weakness-only baseline uses the weakness associated with the candidate's canonical topic.

Sensitivity analysis varies the weakness coefficient between 0.20, 0.35, and 0.50. The remaining four coefficients retain their original relative proportions and sum to the remaining weight. Paired profile-level differences between the default and weakness-only methods are then resampled 20,000 times with replacement to obtain percentile bootstrap intervals.

3.4 Measures and Evaluation Metrics

Classifier accuracy is the number of correct predictions divided by the number of evaluated questions. Macro-F1 gives equal weight to the F1 score of each of the six classes. Reporting the full confusion matrix makes the class support visible,

particularly the fact that Evaluate has only one example. Top-class probability is reported as model confidence; it is not treated as a calibrated probability of correctness.

Precision at five, denoted P@5, is the number of relevant items among the first five recommendations divided by five. With binary relevance, discounted cumulative gain at five is the sum of each top-five item's relevance divided by log2 of its rank plus one. NDCG@5 divides that discounted gain by the gain of an ideal ordering for the same candidate set [18]. The evaluation reports the meaning across the ten profiles for each rater.

Rater diagnostics cover overall quality, weakness fit, curriculum fit, difficulty fit, and clarity. Kappa, exact agreement, agreement within one scale point, and Spearman correlation are used where relevant. Would-use is a separate binary decision. The exact five-point overall kappa must be distinguished from kappa after converting overall ratings to a binary threshold at four.

A metric may be informative without being sufficient. Classifier accuracy does not assess topic assignments, ranking usefulness does not measure learning gains, and agreement between affiliated raters does not establish independent validity. Each result is interpreted only within its corresponding measurement scope.

3.5 Data Analysis

The analysis first describes the evidence available to each subsystem and the distribution of question-bank content. It then presents classifier predictions and ranking performance without combining them into a single overall accuracy value. This separation avoids implying that a correct Bloom label guarantees a useful recommendation.

Ranking comparisons use paired profiles because the competing methods operate on the same profile-level candidate sets. The bootstrap resamples those paired differences rather than treating the 100 candidates as independent estimates of the mean ranking effect. The resulting 95% intervals describe instability within this small pilot sample and are not used to assert population-level statistical significance.

Rater disagreement is interpreted alongside marginal rating distributions. A low kappa can arise when raters use different parts of a scale, whereas low rank correlation can indicate disagreement about ordering. Topic-level failures are also examined against the availability of bank content, since a score formula cannot create a relevant candidate that is absent from the bank.

3.6 Ethics Privacy Security and Responsible AI

The recommendation evaluation uses synthetic weakness profiles and project-affiliated raters. This design does not establish that a student intervention received ethics approval, and no such approval is claimed. Any subsequent study involving real student records, teacher recruitment, or learning outcomes should follow the institution's applicable review and consent processes.

Responsible operation requires limiting student identifiers and raw answer text to what is necessary for the defined educational purpose. Access control, retention limits, and institution-controlled endpoints are relevant design requirements. The paper identifies database credentials in source defaults as a risk requiring removal, rotation, and externally managed secrets; these measures are recommendations until their implementation is demonstrated [2].

Generated questions, topic assignments, and teaching actions require human review. These safeguards are consistent with human-centred guidance for generative AI in education [20]. The presence of a local model does not alone establish privacy, fairness, or reliability. Model outputs and deterministic summaries both require an appropriate institutional context and accountable use.

3.7 Reproducibility and Research Artefacts

The component source is available in the AdaptiveExamAnalyticsEngine directory of the R26-SE-025 repository [3]. The local checkout reference inspected on 9 October 2026 is 636d37767f5ea7880ee63362e6dac038f42de9b9, on branch feat/clean-up-v1.1. This reference identifies the checked-out Git revision, not proof that every local file matches that revision or that the paper evaluated the identical snapshot. File checksums were recorded separately for the local source and evidence inspected during thesis preparation.

The inspected materials include the question bank, the 8,767-row training CSV, two combined rating sheets with 100 rows each, model configuration and weight files, and the test directory. The rating sheets contain ten profiles each. Their would-use counts and major agreement statistics reproduce the corresponding rounded values in the paper. Section 6.6 distinguishes these supplementary checks from the original study.

The available component-level bloom_test_results.json contains eight stored prediction records and is not the paper's eleven-question fixture. It must not be substituted for that fixture. The exact eleven-question input and output record, original baseline/bootstrap script and seeds, and training provenance still need to be associated with the reported numerical account. Appendix A identifies the inspected artifacts and the remaining evidence requirements.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The functional requirements are to ingest graded assessment evidence, assign question semantics, aggregate performance, derive weakness profiles, and rank practice candidates. Outputs must retain enough topic and cognitive context for a lecturer to interpret the recommendation. Optional question generation and teaching-action generation extend the workflow after the core analytics and ranking stages.

The corresponding quality requirements include deterministic numerical calculations, traceable input-output relationships, explicit handling of invalid or unavailable model output, and separation of reported evidence from generated suggestions. A useful educational interface should make the recommendation inspectable and allow a lecturer to reject or revise an unsuitable item.

These requirements describe the intended decision-support role. The case study does not report a validated service-level objective, a formal usability score, or verified production-scale throughput. Such quantities cannot be inferred from the presence of an API and database.

4.2 Component Architecture Within the Complete Solution

The implemented pipeline comprises evidence ingestion, question semantics, deterministic analytics, practice ranking, and optional generation. The FastAPI application in app/main.py registers student and lecturer routes beneath /api and a course-taxonomy router beneath /api/courses. API handlers call service-layer functions, which use the repository layer for MongoDB access [3].

The student pipeline normalizes graded submissions, associates question semantics with the normalized records, computes numeric analytics, and assembles validated student documents. The exam service obtains the course and rubric for the selected assessment, shares a taxonomy snapshot and classification cache across the cohort, computes student-level evidence, and aggregates the results into the lecturer-facing exam document. This structure separates database access, orchestration, and numerical calculations.

The repository layer names separate collections for graded source material and derived outputs. Relevant outputs include student_analytics, studentExamAnalysis, examAnalytics, analyzedExams, generatedQuestions, and exam_drafts. Course taxonomies are stored separately in course_taxonomies. Unique-index definitions express the intended identity of several records, including assessment identifiers and analytics version, although index definitions alone do not prove that indexes were created in a deployed database.

The wider project's lecturer client includes a typed API adapter that requests these analytics, student-detail, recommendation, question-bank, and teaching-action endpoints. It also consumes streamed analysis progress. This establishes a concrete connection in the source between the component and the shared user interface; it does not independently establish a successful live end-to-end execution.

4.3 Design Alternatives and Decisions

A central design choice is to retain deterministic calculations for numerical assessment evidence while restricting probabilistic models to semantics and content generation. This allows the meaning of a reported mark aggregate to remain inspectable, even when the question classifier or generator is uncertain. It also supports separate verification of numerical and model-dependent behavior.

The Bloom classifier is model-based, whereas topic assignment uses DBMS keyword counts and alias normalization. The latter is transparent and inexpensive to inspect, but it can mislabel questions when vocabulary overlaps topics or when important meaning is not expressed through the expected keywords. The paper records topic-label errors and proposes separately evaluated semantic alternatives as future work [2].

The recommendation rule uses hand-set coefficients rather than a trained ranking model. This makes the contribution of each signal visible, but transparency does not establish optimality. The weakness-only comparison is therefore a necessary design check: it tests whether the more complex rule adds useful ordering information under the available evaluation.

The current pipeline refines the paper's topic-matching description with course-local taxonomy matching. Approved lecturer labels are resolved before keyword scoring; exact aliases are distinguished from keywords, archived entries are excluded, and an unresolved tie produces an Unclassified result with a review flag. This observed behaviour should be evaluated in its own right rather than treated as already validated by the paper's historical topic-label findings [3].

4.4 Detailed Design

Each question receives one of six Bloom labels from the local ModernBERT sequence classifier. The service exposes the top-class probability and uses a default input limit of 512 tokens. Topic assignment maps DBMS keyword counts to eleven canonical topics, including SQL, schema refinement, security, JDBC, transactions, recovery, and indexing. Alias maps normalize noisy labels; optional MiniLM embeddings and agglomerative clustering support similarity workflows [2].

Awarded and maximum marks are aggregated across questions, topics, Bloom levels, students, and cohorts. The reported analytics include status bands, item difficulty, top-versus-bottom discrimination, and missed-criterion rates. Topic weakness is the complement of observed achievement. When richer evidence is available, the implementation uses Equation (4.1):

$$W = 0.60(1 - p/100) + 0.25f + 0.15m. \quad (4.1)$$

Here p is average percentage, f is failure rate, and m is missed-criterion rate. The coefficients are engineering priors rather than fitted parameters. The repository provides no independent topic-weakness outcome against which the formula has been calibrated. A low achievement value can therefore contribute to a high weakness score without establishing a validated latent mastery estimate.

For a practice candidate q, the default score is given by Equation (4.2):

$$R(q) = 0.35W + 0.20L + 0.15T + 0.15E + 0.15B. \quad (4.2)$$

The signals are topic weakness W, lecture coverage L, tutorial evidence T, recent-exam relevance E, and a Bloom-level gap B. High and Medium priorities begin at 0.65 and 0.40 respectively. The score is a heuristic ordering quantity, not a

probability that the question will appear in a future exam or that a learner will answer it correctly.

The optional large language model can generate follow-up items after ranking. Schema validation, retries, and deterministic fallbacks constrain degraded operation in the generation workflow. This reported design should not be generalized to the classifier path, where the case study identifies different failure behavior.

The implemented candidate pool normally contains tutorial and generated items. Lecture items are used as candidates only if that pool is empty; lecture presence otherwise contributes a binary coverage signal. Tutorial evidence includes generated items: for one or two such items in a topic, the code uses $0.6 + 0.4(n/3)$, and for three or more it uses 1.0. These details make the rule reproducible and distinguish the actual formula from informal comments about its minimum value [3].

The exam-relevance signal is an inverse frequency signal. The current code selects exam records whose year is at least 2023 and reduces relevance for more frequently represented topics, relative to a normalizing count of at least two. Topics absent from that set receive 1.0. The cutoff is fixed in this implementation, so it should not be described as an automatically rolling two-year window or as a forecast of future content.

Where Bloom-performance evidence is available, the ranker uses one minus average achievement for the candidate's level. Otherwise, it uses a normalized deficit between the observed recent-exam distribution and a target distribution. Equal final scores retain candidate input order under the service's stable descending sort. Reproduction of the paper's rounded ranking values in Section 6.6 instead uses question identifier as an explicit secondary ordering rule.

The richer weakness formula applies when either failure rate or missed-criterion rate is present; a missing component contributes zero. If both are absent, the function returns the percentage complement alone. Inputs to the rate components and the resulting weakness are clamped to the unit interval. These branches need to be recorded when comparing results from analytics records with different evidence completeness.

4.5 Standards and Professional Practice

Professional practice requires a clear contract between documented behavior and implementation. The case study identifies a discrepancy in the Bloom-classification path: documentation describes a 0.60 confidence threshold and fallback, while active service paths trust top-1 output and fail when the model is absent. The design contract should be reconciled with the code before operational reliance is placed on the documented fallback [2].

Traceability also requires documenting which output is a direct calculation, a heuristic score, a semantic prediction, or generated content. For example, a numerical achievement percentage and a model's top-class confidence are both numbers but have different meanings and verification requirements. Exposing those distinctions helps lecturers interpret the system responsibly.

The relevant professional practices include reviewable data transformations, versioned models and datasets, protected credentials, and explicit human oversight. This thesis does not claim certification against software, security, or educational standards. It describes the practices required to develop the prototype into a dependable institutional service.

The inspected exam-analytics code does use the configured confidence threshold to set needs_review. This distinction prevents an overly broad claim that the threshold has no effect anywhere: it can flag uncertain evidence, while the active Bloom service still does not implement the documented threshold-based replacement policy.

4.6 Commercialisation and Sustainability

The prospective value proposition is to help lecturers move from assessment results to reviewable practice recommendations within a course. Potential users include lecturers and academic support staff who already hold graded evidence and maintain course materials. Students may benefit from reviewed practice selected through this process, but the pilot has not measured that benefit or established willingness to pay.

An institution-managed deployment is a plausible sustainability option because the workflow depends on curriculum content, grading evidence, and academic oversight. Maintenance would include model hosting, question-bank curation, integration

support, security, and periodic re-evaluation. These are proposed operating responsibilities; no revenue, cost saving, customer adoption, or commercial agreement is claimed.

Commercial readiness depends on more than technical integration. A future pilot should establish whether lecturers find the explanations useful, how much review effort recommendations require, and whether the service provides value beyond a simpler weakness-only method. Any licensing or ownership arrangement must be resolved with the institution and project contributors before commercialization.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The component is a Python FastAPI service. Its requirements file declares FastAPI 0.141.1, Pydantic 2.13.4, pydantic-settings 2.14.2, PyMongo 4.17.0, Motor 3.7.1, HTTPX 0.28.1, and Transformers 4.55.4. Uvicorn, Torch, and safetensors are also listed without pinned versions. These values describe dependency declarations in the inspected source, rather than independently verified versions of the environment used for the paper [3].

The local classifier configuration identifies ModernBertForSequenceClassification and maps six output labels to BT1-BT6. The loader reads local configuration, safetensors weights, and tokenizer files. It checks for checkpoint key mismatches, sets evaluation mode, and uses inference mode for prediction. The configuration defaults to CPU and a 512-token input limit; the loader can attempt another configured device and report a fallback to CPU if moving the model fails.

The generation configuration defaults to local Ollama with qwen2.5:3b-instruct and provides a separate Colab mode configured for qwen3:8b. The repository also includes switching utilities and notebook instructions. These are configurable operation paths, not evidence that both were used in the paper's experiments. Optional embedding configuration names sentence-transformers/all-MiniLM-L6-v2. The exact research-run environment and hardware remain to be attached to the experiment record.

5.2 Component Implementation

The student pipeline delegates source normalization to app/ingestion/student_data.py and numerical calculation to app/analytics/student_document.py. The latter builds overall, question, topic, and Bloom performance from normalized grading evidence. Qualitative model output is validated separately before selected insight fields are used in the assembled document. This keeps the numerical calculation path distinct from generated learning advice [3].

For uncached questions, the pipeline batches ModernBERT inference and maps the resulting cognition labels into question-semantics objects. It combines a semaphore with an inference lock, and runs inference work outside the event-loop thread. The returned batch length is checked before predictions are associated with questions. These mechanisms describe concurrency control and consistency checks in the implementation; no throughput improvement is claimed without measurement.

Classification caching uses a hash derived from course identity, taxonomy fingerprint, question text, and approved topic, together with the model-version label. The pipeline consults its in-memory cache and, when a database is supplied, bloom_classification_cache. A taxonomy change therefore affects the cache identity. The model-version label is a human-readable string, so a separate checkpoint checksum is still required to establish exact weight identity.

The exam service aggregates canonical-topic evidence using awarded and maximum marks from the same student records used for the class statistics. Unclassified questions remain in overall and Bloom summaries but are excluded from named canonical-topic conclusions. Traceability fields retain Bloom confidence, model version, topic source, and review status. These details explain how the implementation avoids treating an unmatched topic as established subject evidence.

Question generation and teaching actions use the Ollama service and response schemas. Invalid or unavailable generation can produce degraded status, while numeric analysis has deterministic fallback recommendations and generation targets. These fallbacks concern advice generation. The active Bloom path remains dependent on ModernBERT availability and should not be described as having an equivalent rule-based cognition fallback.

5.3 Integration with the Other Components

The component connects to the group solution through graded submissions and shared course and rubric records, then exposes derived analytics through HTTP endpoints. The repository reads submissions and resolves course and rubric references before normalization. It also contains readers for diagram-evaluation and diagram-marking records, while the exam service aggregates those records into a

separate diagram-analysis view. These readers show an implemented integration boundary; they do not transfer ownership of the diagram-evaluation algorithm to this component [3].

The author designed the entire client-side user interface. Its lecturerApi.ts adapter uses VITE_API_BASE_URL (default localhost) for exam, analytics, recommendation, student-detail, teaching-action, and question-bank requests. Streaming requests display analysis progress.

The student router provides profile and dashboard reads, exam listings, and practice-question creation and retrieval beneath /api/students. Lecturer routes additionally support explicit reanalysis and exam-draft creation, retrieval, and deletion. Course-taxonomy routes support reading and maintaining course-specific labels. These are observed API contracts in the source; live authentication, database connectivity, and browser workflows were not exercised during this document review.

The source therefore supports a more specific integration description than the research paper alone. Screenshots and an end-to-end execution record are still required to demonstrate how a real graded submission appears in the interface and leads to reviewed practice output.

5.4 Verification and Software Testing

The inspected test directory contains 41 test files. Their scope includes schemas, ingestion, numerical analytics, recommendation, taxonomy, database repositories, API handlers, and language-model services. The existence and names of these tests establish test assets, not a full-suite pass result. The shared test configuration disables Bloom by default and defines a database fixture whose teardown drops the named test database; the full suite was therefore not executed as part of this thesis preparation [3].

A supplementary offline check directly invoked five existing functions from tests/test_evaluation.py: accuracy, perfect macro-F1, confusion-matrix shape, perfect kappa, and CSV labelling-template creation. All five assertions completed successfully. They were called directly in an isolated output context, without the

pytest runner or repository conftest, and are not represented as a successful complete pytest run.

Eight additional assertion groups checked the actual pure scoring and weakness functions: a known weighted score, priority boundaries, a Bloom-gap calculation, percentage-based weakness, composite weakness, clamping outside the percentage range, empty evidence, and rejection of an out-of-range recommendation input. These checks also completed successfully. They provide narrow arithmetic and boundary evidence, not verification of models, database writes, service endpoints, or user workflows.

Review of the metric helper identified a reproducibility issue: its NDCG function constructs the ideal ranking after truncating relevance to the top k. The standard full-candidate-pool interpretation used in the paper can therefore differ. This discrepancy is illustrated and separated from the reproduced paper values in Section 6.6. The source code was left unchanged during the review.

5.5 Deployment and Operation

The service application entry point is app.main:app. Configuration covers database mode, model location and device, language-model endpoint, timeout, and local or Colab generation selection. The README documents a sample-data workflow that seeds course, rubric, and submission collections and materializes analytics. Because that workflow writes to a database and invokes models, it was inspected rather than executed during thesis preparation [3].

The source includes health endpoints for the language-model backend and Bloom assets, as well as explicit reanalysis and streamed-progress endpoints. The Bloom health response exposes the configured threshold, while actual prediction uses the top class. Exam analytics can mark a result below the threshold for review; this is a review flag and not an alternative classification result.

Operational readiness still requires evidence of deployment configuration, protected credentials, access control, failure recovery, and workload capacity. Available model files establish assets rather than a working service. Production-readiness claims therefore require deployment measurements and a complete environment record.

6 RESULTS AND VALIDATION

6.1 Research Results

The classifier predicts all eleven temporal-fixture labels correctly. Accuracy and macro-F1 across the full six-class label space are 1.000. The ten original fixture items have meant top-class confidence 0.836, with a range of 0.595 to 0.979. The additional Evaluate item is predicted as Evaluate with confidence 0.910 [2]. Table 6.1 presents the reported confusion matrix.

The result gives nominal support for all six classes, but class coverage is uneven: five classes have two examples and Evaluate has one. All prompts are repository-authored and lexically explicit. Thus, the result establishes correct behavior on these eleven cases; it does not estimate robust classification performance across courses or ambiguous questions.

All eleven items have zero exact normalized overlap with the 8,767-question training CSV. This result is limited to the stated comparison procedure and available CSV. Incomplete checkpoint provenance and the absence of a semantic overlap audit remain relevant limitations.

Table 6.1 Temporal fixture confusion matrix with gold labels in rows

Gold	BT1	BT2	BT3	BT4	BT5	BT6
BT1	2	0	0	0	0	0
BT2	0	2	0	0	0	0
BT3	0	0	2	0	0	0
BT4	0	0	0	2	0	0
BT5	0	0	0	0	1	0
BT6	0	0	0	0	0	2

6.2 Product And Technical Results

Table 6.2 reports the ranking comparison. The default rule obtains P@5 of 0.360 and NDCG@5 of 0.420 for lec01, and P@5 of 0.320 and NDCG@5 of 0.414 for lec02.

The corresponding random baseline values are 0.320/0.349 and 0.291/0.357 [2]. These are modest improvements in point estimates within the evaluated profiles.

The weakness-only method obtains P@5 of 0.360 and 0.320, matching the default for both raters. Its NDCG@5 values are 0.436 and 0.429, slightly above the default point estimates. At profile level, the P@5 differences between default and weakness-only are zero for every profile. The paired meaning difference and 95% interval for this metric are therefore exactly zero.

For NDCG, the reported mean default-minus-weakness-only differences are -0.016 and -0.015. Their paired bootstrap 95% intervals are [-0.112, 0.059] and [-0.111, 0.059]. Both intervals include zero. The evidence therefore does not establish that the additional signals improve ranking, nor does it establish that the default is consistently worse in a wider population.

Raising the weakness weight to 0.50 produces NDCG@5 of 0.458 and 0.452, with unchanged P@5. Lowering it to 0.20 gives P@5 of 0.300 and 0.260, and NDCG@5 of 0.320 and 0.314. The higher-weight result is exploratory because the same rating corpus supports its interpretation. It should not be adopted as an independently validated optimum.

Table 6.2 Ranking comparison over ten synthetic weakness profiles

Method	lec01 P@5	lec01 NDCG@5	lec02 P@5	lec02 NDCG@5
Random	0.320	0.349	0.291	0.357
Weakness only	0.360	0.436	0.320	0.429
W = 0.20	0.300	0.320	0.260	0.314
Default W = 0.35	0.360	0.420	0.320	0.414
W = 0.50	0.360	0.458	0.320	0.452

6.3 User Or Stakeholder Validation

The two raters mark 32% and 29% of candidates as items they would use. Kappa for would-use is 0.882. Kappa is also 0.882 for overall ratings dichotomized at four,

whereas the exact five-point overall kappa is 0.744. These values concern different representations of the ratings and should not be conflated [2].

Weakness-fit kappa is 0.208, yet Spearman correlation is 0.891 and 95% of rating pairs are within one point. The marginal distributions explain much of this difference: lec01 assigns score 1 to 68 items, while lec02 assigns score 2 to 57. The raters largely order candidates similarly but use different scale anchors.

Difficulty-fit is less consistent. Kappa is -0.088, exact agreement is 48%, within-one-point agreement is 98%, and Spearman correlation is -0.029. Lec01 gives score 4 to 94 of the 100 items, while lec02 uses scores from 2 to 5. Marginal concentration affects kappa, but the near-zero rank association also indicates that the raters do not share a reliable ordering of difficulty fit.

These results are internal pilot validation of perceived usefulness and measurement consistency. They do not constitute independent lecturer validation, a usability trial, or evidence that students learn more effectively after receiving the recommendations.

6.4 Integration Results

The inspected source provides component-level integration evidence through shared submission, course, and rubric readers; diagram-analysis inputs; persistent analytics outputs; and matching calls in the lecturer client [3]. The API adapter and backend routes agree on the principal analytics and recommendation paths. These observations establish intended connections in code, not a measured outcome from running the complete group system.

The current exam service also preserves review flags for low Bloom confidence and uncertain topic assignment. This adds nuance to the paper's fallback discussion: a low-confidence prediction can be flagged, but the active classifier still retains the ModernBERT top-class result rather than substituting a rules or language-model classification. The original classification and ranking findings remain scoped to the case study, rather than to validation of all group components.

6.5 Summary of Findings

The reported evidence supports three main findings. First, the deployed classifier behaves correctly on a small six-class fixture. Second, the additional signals in the default ranker do not establish incremental value over weakness alone in the ten-profile evaluation. Third, content coverage and rater calibration constrain the usefulness and interpretation of the recommendations.

Question-bank coverage helps explain topic-level failure. SQL profiles attain P@5 of 0.80 and 0.60, while JDBC attains 0.60 for both raters. Transaction and utility profiles have P@5 of 0.00 for both raters, and recovery has 0.00 and 0.20. Only three bank records cover each of indexing, transactions, and recovery [2]. The reported coverage distribution is summarized in Table 6.3.

Table 6.3 Question bank distribution by source and Bloom level

Dimension	Category	Records	Share
Source	Lecture	90	50.3%
Source	Tutorial	55	30.7%
Source	Generated	18	10.1%
Source	Prior exam	16	8.9%
Bloom	Remember	0	0.0%
Bloom	Understand	39	21.8%
Bloom	Apply	132	73.7%
Bloom	Analyze	8	4.5%
Bloom	Evaluate	0	0.0%
Bloom	Create	0	0.0%

6.6 Supplementary Repository Verification

A supplementary offline check on 9 October 2026 read the two combined_100 rating sheets and the question bank from the local component snapshot [3]. Each rating sheet contains 100 rows across ten profiles. The bank contains 179 records, with

source and Bloom distributions matching Table 6.3. The training CSV contains 8,767 rows. These are direct artifact checks rather than a new learner study.

For each profile, the supplementary calculation uses binary would-use relevance, sorts by descending recommendation score with question identifier ascending for ties, and computes ideal discounted gain from the full profile candidate pool. Weakness-only ranking uses the weakness-context value for the candidate topic and the same secondary ordering rule. This procedure reproduces the paper's rounded default and weakness-only P@5 and NDCG@5 values. It establishes a reproducible ordering convention consistent with the reported aggregates, not proof that the unavailable original experiment script used that exact convention.

The recalculated default NDCG@5 values are 0.419771 and 0.414063; the weakness-only values are 0.436072 and 0.429462. P@5 remains 0.360 and 0.320 for both methods. With CSV order retained for ties instead, default NDCG@5 is 0.412770 and 0.420787. This sensitivity shows why the candidate-order policy belongs in the reproducibility record.

The would-use, exact overall, weakness-fit, and difficulty-fit kappa values also match the paper after rounding: 0.882, 0.744, 0.208, and -0.088. Before pairing the two sheets, profile identifiers, question identifiers, and question text were checked row by row. Repeated identifiers were disambiguated by occurrence so that all 100 rows were preserved.

The repository's general ndcg_at_k helper differs from the full-pool calculation. For relevance [true, false, false, false, false, true] at k = 5, the helper returns 1.000 because it ignores the relevant sixth item when constructing the ideal ranking. The full-pool calculation returns approximately 0.613147. The reproduction above uses the latter definition, consistent with the method described in the paper; it is not a claim that the repository helper has been corrected.

The checks in Section 5.4 passed without database fixtures, live services, browser workflows, or model inference. Random-baseline and bootstrap experiments were not rerun. The classifier, random-baseline, and uncertainty findings therefore retain their original reported status until the exact execution artifacts are attached.

7 DISCUSSIONS

7.1 Interpretation of Findings

The findings indicate that the prototype's strongest defensible contribution is the organization and evaluation of a complete evidence-to-action workflow. The results do not support treating classifier success, recommendation quality, and educational benefit as interchangeable. Each requires distinct data and a different form of validation.

The eleven-question classifier result addresses a limited structural gap by including every Bloom class. It does not remove uncertainty about implicit cognitive demands, ambiguous wording, or within-class variation. A model can classify deliberately explicit prompts correctly while struggling with authentic questions whose cognitive demand depends on the expected answer or marking rubric.

The ranking comparison changes how the five-signal rule should be interpreted. Its additional contextual terms are understandable design choices, but their combined inclusion has not demonstrated a measurable advantage over the strongest simple baseline in this pilot. This finding supports further evaluation and calibration rather than a claim that complexity itself improves personalization.

7.2 Comparison with Literature Baselines and Existing Solutions

The system is consistent with the broad aim of adaptive educational recommendation: connect learner information with suitable resources [4], [5]. Its deterministic weakness snapshot and fixed ranking rule differ from sequence-based knowledge tracing and learned temporal state representations [8], [9]. Therefore, the contribution is integration and auditability rather than a new student-state estimation method.

The baseline evidence suggests a general evaluation lesson. A hand-tuned multi-signal heuristic should be compared with its strongest simple component before its extra terms receive credit. A random baseline establishes a low reference point, but a weakness-only baseline is more informative here because it directly tests the value of the additional curriculum and cognitive signals.

NDCG and precision reveal complementary aspects of the outcome [18]. The default and weakness-only methods have identical precision in each evaluated profile, while ordering differences affect NDCG. This indicates that any contribution from the added terms should be investigated at the level of ranking, order and relevance, rather than inferred from the number of signals used.

7.3 Research Contribution

The research contribution is a reproducible-evaluation orientation for a concrete educational software artifact. The study brings classifier coverage, recommendation baselines, paired uncertainty, sensitivity, and rater diagnostics into a single assessment of the component [2]. It also identifies what each method can and cannot establish.

The rater analysis provides a particularly useful methodological distinction. Weakness-fit disagreement largely reflects scale anchoring despite high ordinal association. Difficulty-fit disagreement includes near-zero ordinal association as well as concentrated marginal ratings. These patterns require different remedies: anchor calibration may help the former, whereas the latter also requires a clearer shared definition of the construct.

The study consequently contributes failure analysis and evaluation lessons rather than evidence of a new optimal formula. Its findings are useful because they identify limits that would otherwise remain hidden behind a working demonstration.

7.4 Product Contribution

The product contribution is a component that connects question semantics, deterministic assessment summaries, weakness scores, ranked practice, and optional generated actions within one service. Canonical topics and cognitive labels provide a shared representation through which lecturers can inspect the relationship between evidence and recommended practice.

The design also exposes opportunities for accountable operation. Marks can remain traceable to input evidence, ranking coefficients can be inspected, and generated suggestions can be reviewed before use. These are useful product properties,

although their presence does not establish measured usability, user acceptance, or production reliability.

The immediate development priorities follow from the observed failures: reconcile classifier fallback behavior, correct topic-label errors, improve question-bank coverage, and strengthen the human evaluation rubric. Adding more model complexity before addressing those issues would not resolve the missing-content problem.

7.5 Limitations and Threats to Validity

Construct validity is limited using perceived usefulness as the principal recommendation outcome and by inconsistent interpretation of difficulty fit. A would-use decision is meaningful for candidate selection, but it is not a measure of learning gains. The weakness formula also lacks an independent outcome against which its coefficients can be calibrated.

Internal validity is constrained by hand-set weights, post hoc sensitivity analysis, and interpretation based on the same rating corpus. The default-minus-baseline intervals are wide, and only ten profiles are resampled. The bootstrap represents pilot-sample variability rather than resolving the limitations of the sampling design.

External validity is limited by one DBMS course, repository-authored classifier examples, synthetic weakness profiles, 100 rated candidates, and two affiliated raters. The single Evaluate item improves nominal class support without supplying independent validation. The findings may not transfer to different subjects, institutions, learner populations, or authentic assessment conditions.

Data and implementation validity are affected by incomplete checkpoint-training provenance, keyword topic errors, and substantial question-bank imbalance. Zero exact overlap does not exclude semantic leakage. A confidence value is not proven calibration, and the documented fallback contract does not match active classifier behavior. Together, these limitations support treating the system as a pilot prototype.

The supplementary repository review finds 100 rating rows but only 91 distinct combinations of profile and question identifier. Repeated identifiers were retained to reproduce the reported row-based results, rather than silently deduplicated. Future

evaluation should establish whether these records represent deliberate repeated candidates, distinct content sharing an identifier, or duplicate rows, and should define candidate identity and tie handling before scoring. The reproduced aggregates do not resolve that data-design question.

7.6 Implications

For research, the next evaluation should separate development and test profiles, use independently authored Bloom examples, and compare with stronger lexical, embedding, and learned ranking baselines where sufficient data exist. Weight tuning should occur only on development data. An external rater should assess a preselected subset to examine whether the rubric transfers beyond the project team.

For practice, question-bank expansion and quality review are prerequisites for broader use. No scoring adjustment can retrieve a suitable item from an empty or mislabeled part of the bank. Balanced topic coverage and reviewed higher-order questions are therefore at least as important as changes to the ranking formula.

Difficulty fit should be defined as the relationship between expected item difficulty and a specified target challenge band. Anchored examples, rather training, and adjudication of a calibration subset would make the intended interpretation clearer. Only after improving content and evaluation quality should a learning-outcome study test whether recommendations benefit students.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This thesis examines an AI-assisted exam analytics and weakness-aligned practice recommendation component within a wider academic evaluation project. The case study shows how question semantics and deterministic assessment summaries can be connected to a transparent practice-ranking workflow. Its implemented scope is analysis and recommendation rather than forecasting examination content.

The deployed classifier returns the expected labels for all eleven questions in a small temporal fixture, but this result remains limited integration evidence. The default ranker has modestly higher point estimates than random ordering, while weakness-only ranking matches its precision and has slightly higher NDCG point estimates. Paired intervals cross zero, so the four additional signals have not established incremental value within the available profiles.

The main conclusion is that transparent architecture must be accompanied by transparent evaluation. Baselines, content-coverage analysis, and rater diagnostics reveal limitations that a successful end-to-end demonstration would not identify. The prototype offers a basis for further research, while wider educational effectiveness and production readiness remain to be demonstrated.

8.2 Achievement of Objectives

The architecture objective is addressed through the documented separation of evidence ingestion, semantic interpretation, numerical analytics, ranking, and optional generation. The classifier objective is addressed provisionally through six-class fixture coverage and an exact normalized overlap check, with the limitations of sample size and provenance retained.

The ranking comparison objective is addressed through random and weakness-only baselines, P@5, NDCG@5, paired bootstrap intervals, and the sensitivity experiment. The outcome is not a confirmed improvement for the multi-signal rule; it is an evidence-based limit on that claim. The diagnostics objective is addressed by

the analysis of scale calibration, difficulty-fit disagreement, topic coverage, and service-contract inconsistency.

The study does not claim achievement of objectives that were not tested. It does not validate a future-exam predictor, demonstrate improved student attainment, or establish independent cross-institutional performance. These boundaries are part of the interpretation of the completed case study.

8.3 Individual Research And Product Contributions

The individual component's research contribution is the characterization and evaluation of an assessment-to-practice pipeline. Its product contribution is the integrated handling of question semantics, assessment aggregation, weakness representation, and practice ranking, together with optional generation support [2].

The documented evaluation contributes a six-class temporal fixture result, recommendation baselines, uncertainty estimates, sensitivity analysis, and rater-calibration findings. These define the component's current capabilities and the evidence needed for stronger educational claims. The author developed the complete AdaptiveExamAnalyticsEngine component, carried out its documented research, and designed the entire client-side user interface for the group project.

8.4 Future Directions

The priority is an independently authored and leakage-audited Bloom evaluation set with substantial support for all six classes, including questions whose cognitive demands are not stated through explicit cue words. Checkpoint provenance should be recorded, and confidence should be calibrated before a threshold is used as an operational decision rule.

The second priority is to improve the practice bank. Topic coverage should be expanded for indexing, transactions, recovery, and other sparse areas, while cognitive coverage should include reviewed Remember, Evaluate, and Create items. Topic-label quality must be checked because incorrect labels undermine both weakness matching and evaluation.

The third priority is a stronger recommendation study. Separate development and test profiles should support weight selection and unbiased evaluation. Baselines should include suitable lexical and semantic methods, and learned ranking should be considered only when data support training and evaluation. A revised difficulty rubric and at least one external rater should strengthen human judgement quality.

The fourth priority is evidence of operational and educational value. Integration tests, failure-path tests, and measured runtime behavior should precede deployment claims. A subsequent student study could assess learning outcomes using an appropriately reviewed design, but it should begin only after the content, recommendation process, and human-review workflow are sufficiently reliable.

REFERENCES

[1] G. Siemens, "Learning analytics: The emergence of a discipline," American Behavioral Scientist, vol. 57, no. 10, pp. 1380-1400, 2013, doi: 10.1177/0002764213498851.

[2] L. S. B. Hemarathne, "AI-Assisted Exam Question Analytics and Weakness-Aligned Practice Recommendation: A Repository Case Study," research manuscript supplied by the author, 2026.

[3] R26-SE-025 project contributors, "AdaptiveExamAnalyticsEngine," R26-SE-025 source repository, local working copy reviewed Oct. 9, 2026. [Online]. Available: https://github.com/akilaManu-MaHiTo/R26-SE-025/tree/feat/clean-up-v1.1/AdaptiveExamAnalyticsEngine

[4] P. Brusilovsky and C. Peylo, "Adaptive and intelligent Web-based educational systems," Int. J. Artif. Intell. Educ., vol. 13, no. 2-4, pp. 159-172, 2003, doi: 10.3233/IRG-2003-13(2-4)02.

[5] K. Verbert, N. Manouselis, X. Ochoa, M. Wolpers, H. Drachsler, I. Bosnic, and E. Duval, "Context-aware recommender systems for learning: A survey and future challenges," IEEE Trans. Learn. Technol., vol. 5, no. 4, pp. 318-335, 2012, doi: 10.1109/TLT.2012.11.

[6] J. K. Tarus, Z. Niu, and G. Mustafa, "Knowledge-based recommendation: A review of ontology-based recommender systems for e-learning," Artif. Intell. Rev., vol. 50, pp. 21-48, 2018, doi: 10.1007/s10462-017-9539-5.

[7] J. K. Tarus, Z. Niu, and A. Yousif, "A hybrid knowledge-based recommender system for e-learning based on ontology and sequential pattern mining," Future Gener. Comput. Syst., vol. 72, pp. 37-48, 2017, doi: 10.1016/j.future.2017.02.049.

[8] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," User Model. User-Adapt. Interact., vol. 4, pp. 253-278, 1995, doi: 10.1007/BF01099821.

[9] C. Piech et al., "Deep knowledge tracing," in Advances in Neural Information Processing Systems 28, 2015, pp. 505-513.

[10] L. W. Anderson and D. R. Krathwohl, Eds., A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives. New York, NY, USA: Longman, 2001.

[11] E. Aeiad and F. Meziane, "An adaptable and personalised e-learning system applied to computer science programmes design," Educ. Inf. Technol., vol. 24, no. 2, pp. 1485-1509, 2019, doi: 10.1007/s10639-018-9836-x.

[12] A. C. Graesser et al., "ElectronixTutor: An intelligent tutoring system with multiple learning resources for electronics," Int. J. STEM Educ., vol. 5, art. 15, 2018, doi: 10.1186/s40594-018-0110-y.

[13] A. Klasnja-Milicevic, M. Ivanovic, B. Vesin, and Z. Budimac, "Enhancing e-learning systems with personalized recommendation based on collaborative tagging techniques," Appl. Intell., vol. 48, no. 6, pp. 1519-1535, 2018, doi: 10.1007/s10489-017-1051-8.

[14] B. Warner et al., "Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference," arXiv:2412.13663, 2024.

[15] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in Proc. EMNLP-IJCNLP, Hong Kong, China, 2019, pp. 3982-3992, doi: 10.18653/v1/D19-1410.

[16] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, "A systematic review of automatic question generation for educational purposes," Int. J. Artif. Intell. Educ., vol. 30, pp. 121-204, 2020, doi: 10.1007/s40593-019-00186-y.

[17] S. Elkins, E. Kochmar, J. C. K. Cheung, and I. Serban, "How teachers can use large language models and Bloom's taxonomy to create educational quizzes," arXiv:2401.05914, 2024.

[18] K. Jarvelin and J. Kekalainen, "Cumulated gain-based evaluation of IR techniques," ACM Trans. Inf. Syst., vol. 20, no. 4, pp. 422-446, 2002, doi: 10.1145/582415.582418.

[19] J. Cohen, "A coefficient of agreement for nominal scales," Educational and Psychological Measurement, vol. 20, no. 1, pp. 37-46, 1960, doi: 10.1177/001316446002000104.

[20] F. Miao and W. Holmes, Guidance for Generative AI in Education and Research. Paris, France: UNESCO, 2023.

GLOSSARY

Canonical topic. A normalized course-topic label used consistently across analytics and recommendation.

Cognitive demand. The thinking process a question requires, represented here through revised Bloom categories.

Heuristic ranker. A candidate-ordering method that applies a fixed scoring rule rather than learning its parameters from outcomes.

Temporal fixture. A set of repository examples dated after the deployed checkpoint and used for a limited integration check.

Weakness profile. A summary of learning needs derived from observed assessment evidence.

Would-use decision. A rater judgement that a candidate question would be suitable for use in the stated profile.

APPENDICES

Appendix A Evaluation Evidence Record

The inspected component is AdaptiveExamAnalyticsEngine in the R26-SE-025 repository [3]. The local source review was conducted on 9 October 2026. The checked-out revision reference is 636d37767f5ea7880ee63362e6dac038f42de9b9; the local file inventory records SHA-256 checksums independently of that reference. The source review does not establish that this is the exact snapshot used for the paper.

Implementation evidence: app/main.py; app/api/dashboard.py and lecturer.py; app/services/student_pipeline.py, exam_analytics.py, recommendation.py, and llm_service.py; app/analytics/weakness.py and recommendation_score.py; app/taxonomy/matching.py; app/db/repository.py; and requirements.txt. The shared client connection was inspected in Gradex_AI_Client/src/app/api/lecturerApi.ts.

Evaluation evidence: datasets/bloom_dataset/question_bank.json; combined_100_lec01.csv and combined_100_lec02.csv in the same directory; datasets/bloom_dataset/bloom/blooms_taxonomy_dataset.csv; models/bloom_modernbert/config.json and bloom.safetensors; app/evaluation/metrics.py; and tests/test_evaluation.py. File locations identify artifacts rather than implying that every stored item was used in the reported experiment.

Supplementary verification: five existing pure test functions and eight arithmetic/boundary assertion groups completed successfully. The rating-sheet reconstruction used a full-pool ideal ranking and question-ID tie breaking, preserving all 100 candidate rows per rater. Section 6.6 reports the outcome and limits of these checks.

Appendix B Academic Integrity and AI Use Disclosure

Generative AI assistance through OpenAI Codex was used to structure and draft thesis text from the author-supplied research manuscript, template, and component source; apply formatting; inspect repository evidence; and prepare supplementary offline calculations and pure-function checks. No new participant data were collected and the original model-inference, random-baseline, and bootstrap experiments were not rerun. The supplementary checks and their limitations are reported separately in Sections 5.4 and 6.6. The original research manuscript is reference [2].